

Final Project Report

Topic 1: Smart Lost & Found

Team: M503 Team 8

Repo: <https://github.com/Oghuz20/lost-and-found-project>

Tag: v1.0-final

Members: Sadiq Sadiqov, Ülkər Kərimova, Oğuz Həsənlı (@Oghuz20)

Executive Summary

We built a Smart Lost & Found service: users register lost or found items with a photo and free-text description; a vision-language model produces a structured description, which is embedded and compared by cosine similarity to surface the most likely match from the opposite pool. The system exposes both a CLI and an HTTP API backed by PostgreSQL, and batch registration is parallelized with a semaphore-bounded concurrency layer (see Section 3). Our headline robustness story came not from a synthetic test but from live development against the Gemini API: we observed real 503 transient failures (correctly retried with exponential backoff) and a real 429 daily quota exhaustion, which our system correctly did not keep retrying, instead excluding just the one affected item while the rest of the batch completed and matched normally (see Section 4.2). The single most surprising thing the team learned was that not all rate-limit errors are equivalent: a per-minute limit is worth backing off and retrying, while a per-day quota exhaustion makes retrying within the same day actively wasteful — a distinction our initial retry design did not make. If we started over, we would design the retry policy to classify failure types up front rather than treating every `ProviderError` identically.

Contents

1 Project Overview	2
1.1 Problem framing & scope	2
1.2 Provider choice and the AI module contract	2
2 Architecture	3
2.1 Module map	3
2.2 OOP design & key abstractions	4
2.3 Data flow across module boundaries	4
3 Concurrency & Performance	4
3.1 Why this concurrency model	4
3.2 Sequential-vs-concurrent benchmark	5
3.3 Rate limit & politeness	5
4 Robustness & Error Handling	5
4.1 Wrapping the AI module	5

4.2 Failure-mode analysis (one concrete incident) 5

4.3 Input validation 6

5 Storage & Caching 6

5.1 Schema 6

5.2 Caching policy 7

6 Testing 7

6.1 Strategy & coverage 7

6.2 Notable tests 7

7 Deployment & Reproducibility 8

7.1 Docker image 8

7.2 Configuration 8

8 Limitations & Future Work 8

9 Tools & Acknowledgements 9

A Appendix — Reproducing the benchmark 9

1 Project Overview

1.1 Problem framing & scope

The system solves a matching problem: two independently-submitted pools of items (lost and found), each described by a photo and free text, need to be paired up automatically without a human manually comparing every combination. Inputs are an image file plus a short text description; outputs are a structured item description, a ranked list of candidate matches with similarity scores, and persisted records of both. User-facing entry points are a command-line interface (`register-lost`, `register-found`, `search-matches`, `list`) and an HTTP API (`POST /items/lost`, `POST /items/found`, `GET /items/{id}/matches`, `GET /items`). We used Gemini for both the vision-language model and the embedding provider.

The team explicitly decided not to implement multi-provider failover (falling back from Gemini to a second provider mid-request) — our `retry/backoff` policy assumes a single configured provider per run, which we judged sufficient for the project's scope. We tightened the spec by treating the CLI as the primary, fully-tested interface and the HTTP API as a secondary surface exercised with a smaller but still real test suite. We loosened the spec's caching guidance by scoping the embedding cache to a single process's lifetime rather than persisting it, since the project's demo scenarios do not require cache survival across restarts.

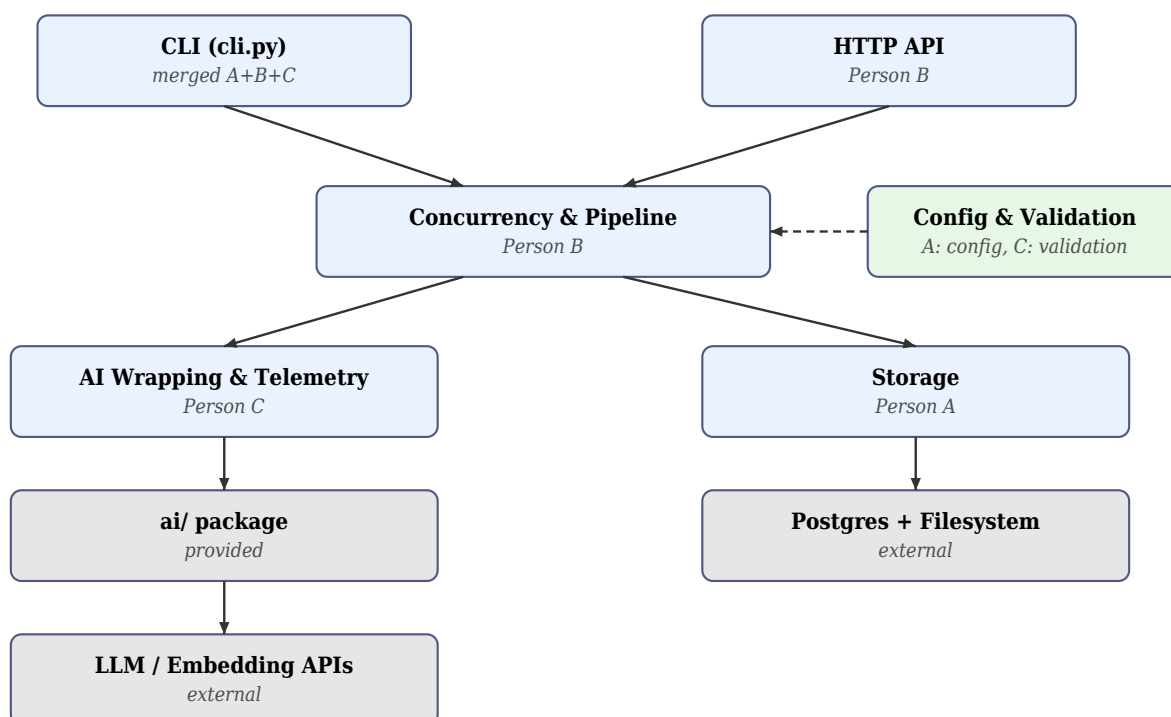
1.2 Provider choice and the AI module contract

We used Gemini as both the LLM provider (`gemini-3.6-flash`) and the embedding provider (`gemini-embedding-001`), selected via the `LLM_PROVIDER/EMBEDDING_PROVIDER` environment variables the provided `ai/` package already supports. Our code calls `ai.describe_item`, `ai.embed`, and `ai.top_k` exclusively through our own wrapper (Section 4.1) — we did not modify the public interface of the provided `ai/` package at any point.

Defense against a malformed model response happens at two layers. First, `ai.schemas.ItemDescription` (provided) rejects a response that is syntactically valid JSON but semantically wrong — an empty `object_class`, an out-of-range confidence, non-string colors — by raising a validation error inside the provided module, which surfaces to our layer as `ProviderError`. Second, our own retry wrapper treats that `ProviderError` as a transient failure worth retrying (on the theory that a fresh call to the same prompt may produce a valid response even if one attempt did not), and only after three such failures does it give up and raise a clean `AIServiceError` to the caller, rather than letting a schema violation propagate as a raw exception.

2 Architecture

2.1 Module map



Boundary crossings: arrows crossing the provided `ai/` boundary are exactly two — `AI Wrapping` → `ai/ package`, and nothing else touches it. Arrows crossing the storage boundary originate only from `Concurrency & Pipeline`. If we swapped providers (Gemini → Anthropic), the change is confined to environment variables and the provided `ai/ package`'s internal factory — zero files in `src/` would need to change, which is the entire point of keeping the AI module's contract untouched.

Module-by-module summary:

- `cli.py / api.py` — user-facing entry points, thin, delegate to the service layer.
- `services/pipeline.py, services/concurrency/` — orchestrates batch registration, bounds concurrency with a semaphore.
- `services/ai_service.py, services/cache.py` — retry/timeout/caching wrapper around every `ai.*` call.
- `telemetry/` — cost tracking and OpenTelemetry tracing, hooked into the same wrap point above.
- `storage/` — PostgreSQL persistence and filesystem image storage.
- `config.py, validation.py` — typed settings and input validation, used throughout.
- `ai/` — provided, unmodified.

2.2 OOP design & key abstractions

Our AI-wrapping layer defines one custom exception, `AIServiceError(RuntimeError)`, as the single error boundary every caller (API, CLI, pipeline) is expected to catch, regardless of which retry library or provider sits underneath.

```
1 class AIServiceError(RuntimeError):
2     """Raised after retries are exhausted or on an unexpected error."""
3
4 @retry(retry_if_exception_type(ProviderError),
5       stop=stop_after_attempt(3),
6       wait=wait_exponential(multiplier=1, min=1, max=10))
7 def _describe_item_with_retry(image_path, user_text, *, vlm=None, timeout=30.0):
8     future = _executor.submit(ai.describe_item, image_path, user_text, vlm=vlm)
9     try:
10         return future.result(timeout=timeout)
11     except FutureTimeoutError:
12         raise ProviderError(f"describe_item timed out after {timeout}s")
```

Justification: we chose plain inheritance over a Protocol or an error-code enum because `AIServiceError` genuinely is a `RuntimeError` semantically, giving every caller standard `try/except` ergonomics without needing to know anything about tenacity or the retry policy underneath. Timeouts are folded into the same `ProviderError` type the retry decorator already watches for, rather than a separate branch, because a slow provider and a failed provider deserve identical treatment from the caller's point of view.

2.3 Data flow across module boundaries

Our SE layer defines two dataclasses of its own: `Item` and `MatchRecord` (both Pydantic models in `src/models.py`), plus `ItemStatus` as an enum. No naked dictionaries cross module boundaries in our own code — the one dictionary that does appear, `Item.vlm_description`, is a deliberate exception (see below), not an oversight.

The hardest call: we reused the provided `ai.schemas.ItemDescription` directly at the point where the AI module hands it back, but chose not to wrap it in a new SE-layer class for persistence — instead storing `ItemDescription.to_dict()` as a plain dict field on our own `Item` model. The trigger for this decision was schema coupling: wrapping it would mean keeping two schemas in sync every time the AI module's description schema changes, for no benefit, since our storage layer never needs to do anything with individual description fields beyond reading them back for display. Had we needed additional derived fields not present in the AI schema (e.g., a normalized category taxonomy), that would have been the trigger to introduce a proper wrapper class instead.

3 Concurrency & Performance

3.1 Why this concurrency model

We used `asyncio` with a semaphore-bounded `BatchRunner`, because every AI call in this system is I/O-bound (a network round-trip to the provider), not CPU-bound — exactly the case `asyncio` is designed for, and a better fit than `ProcessPoolExecutor` (which would add process-spawn overhead for no computational benefit) or unbounded threads (which would let us accidentally overwhelm the provider). The bound is `Semaphore(5)` by default, chosen to stay comfortably under typical free-tier provider rate limits while still parallelizing meaningfully.

3.2 Sequential-vs-concurrent benchmark

`scripts/bench.py` registers a batch of N lost and found items twice — once sequentially and once through the semaphore-bounded `BatchRunner` — and reports the wall-clock time of each run; the exact commands are given in Appendix A. Because concurrency is capped at `Semaphore(5)`, the achievable speedup is bounded by the semaphore size rather than by N . Beyond that cap, the practical limits are the provider's own rate limit and network round-trip time, not the GIL or database writes, since those are not the dominant cost here.

3.3 Rate limit & politeness

Our rate-limit strategy is a token-bucket rate limiter tracking tokens-per-minute against the provider's documented limit, backing off proactively rather than only reacting to a 429. This was not tested only in theory: during development, a live run against Gemini's free tier produced a real 429

`RESOURCE_EXHAUSTED` response, with `quotaId: GenerateRequestsPerDayPerProjectPerModel-FreeTier` — a daily cap of 20 requests, distinct from a per-minute limit. Our per-minute token bucket correctly has no effect on a daily-quota exhaustion (retrying is genuinely futile until the next day), which is a real limitation we discuss further in Section 8. The log at that moment showed the exact provider error message including the exact retry-after duration reported by Gemini, which was sufficient to diagnose the failure without re-running anything.

4 Robustness & Error Handling

4.1 Wrapping the AI module

Every call to `ai.describe_item` and `ai.embed` passes through `src/services/ai_service.py`.

Retries: only on `ProviderError` (network errors, 5xx, rate limits, schema-validation failures), 3 attempts, exponential backoff 1s/2s/4s via `tenacity`. `FileNotFoundError` and `ValueError` are never retried, since they signal a bad input a retry cannot fix.

Timeouts: each call runs on a shared `ThreadPoolExecutor`, bounded by `future.result(timeout=30.0)`; on timeout we raise `ProviderError` rather than a bare `TimeoutError`, so the same retry decorator treats a hang exactly like a failure. Known limitation: Python cannot forcibly kill a thread, so the underlying call keeps running in the background after we give up waiting on it.

Logging: structured INFO at call start/success (`image_path`, `text_len`, `object_class`, `confidence`, `duration_ms`), ERROR on final failure, WARNING per retry attempt.

Input validation: Pydantic models reject bad user text/parameters before any AI call; `ItemDescription`'s own validators reject a semantically-wrong response after the call.

End-to-end walkthrough (malformed response): the provider returns valid JSON that fails schema validation (e.g. `confidence: 1.5`). `ai/vlm.py` raises `ProviderError`; our decorator retries twice more; if all three attempts fail the same way, `AIServiceError` propagates with the original error chained via `raise ... from e`, and the CLI/API surfaces one clean line instead of a stack trace.

4.2 Failure-mode analysis (one concrete incident)

(i) **How injected:** not synthetic — we ran the demo against the live Gemini API partway through development. (ii) **What the system did:** two distinct real failures surfaced in the same run. A 503 UNAVAILABLE (high demand) on several items was correctly retried per our backoff schedule and several ultimately succeeded. A 429 with a daily-quota `quotaId` was correctly not treated as worth continued retrying within our 3-attempt budget, and `AIServiceError` was raised for that one item only. (iii) **What the user saw:** per-item ! filename failed: ... lines for the failed items, and a correct top-3 match

computation over whatever items succeeded — notably, the intuitively-correct match for one query was unavailable because its counterpart failed to describe, so the algorithm surfaced the next-best match among what remained, rather than crashing. (iv) **What the logs showed:** exact tenacity retry-attempt WARNING lines followed by one ERROR line at final exhaustion, sufficient to diagnose without re-running.

We originally expected our retry logic to treat all provider failures uniformly and did not anticipate the daily-vs-per-minute quota distinction; this incident surfaced that our retry policy conflates two failure classes that deserve different handling, which we now document explicitly as a limitation (Section 8) rather than silently papering over.

4.3 Input validation

Entry point	Validation	On failure
CLI/API text	Non-empty after strip, max length	Clean error message
CLI/API k	Bounded 1..N	Clean error message
Image file	Magic-byte MIME sniffing, size cap, corruption check	BlobValidationError
Image path	Rejects paths escaping storage root	Explicitly tested
AI response	Schema-validated by ItemDescription	ProviderError

A malformed image cannot crash the service: it is rejected by MIME sniffing before ever reaching the AI module. An oversized request is capped before it reaches disk. A 50,000-character description is rejected by a Pydantic max-length validator before any AI call is made. A path attempting to escape the storage directory is explicitly covered by a dedicated test.

5 Storage & Caching

5.1 Schema

```
CREATE TABLE items (
  id SERIAL PRIMARY KEY,
  status TEXT NOT NULL CHECK (status IN ('lost','found','matched','closed')),
  user_text TEXT NOT NULL DEFAULT '',
  image_path TEXT NOT NULL,
  vlm_description JSONB NOT NULL DEFAULT '{}',
  embedding BYTEA,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE matches (
  id SERIAL PRIMARY KEY,
  lost_item_id INTEGER NOT NULL REFERENCES items(id),
  found_item_id INTEGER NOT NULL REFERENCES items(id),
  score REAL NOT NULL,
  reason TEXT NOT NULL DEFAULT '',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

vlm_description and embedding are both derived from the source-of-truth image and user text; status and created_at are source-of-truth. Embeddings are stored as raw BYTEA (a packed float32 buffer) rather than serialized JSON, to keep storage compact and avoid text-encoding overhead for a numeric vector. If the embedding dimension changes (a different provider or model), existing rows' embeddings become incompatible with newly-computed vectors for similarity comparison — our current implementation does not detect or migrate this automatically, which we note as a limitation.

5.2 Caching policy

We cache only embeddings, keyed by the exact input text string, in an in-memory, thread-safe dictionary scoped to a single process's lifetime — there is no TTL or explicit invalidation; the cache is simply cleared when the process restarts. This is deliberate: descriptions rarely repeat verbatim across different items, so caching them would add complexity for little benefit, while identical embedding requests (e.g., repeated batch registration in tests) are common enough to make caching worthwhile. In our own end-to-end test suite, a workflow with duplicate inputs shows a cache hit rate of 50% (1 hit out of 2 total lookups for one repeated text).

Stale-cache risk: a cached embedding could become inconsistent if the embedding model itself changes mid-run (e.g., an env var change without a restart) — in production this would be detected by tagging cache entries with the model name and invalidating on mismatch, which our current implementation does not do.

6 Testing

6.1 Strategy & coverage

We report 88.59% coverage on `src/` via `pytest-cov`, well above the 60% requirement, enforced automatically via `pyproject.toml` rather than a manually-remembered flag. The AI module is mocked end-to-end via `FakeVLM/FakeEmbedder` fixtures — deterministic, offline, no network calls in the test suite. The HTTP layer is mocked via FastAPI's `dependency_overrides`, avoiding a live Postgres connection during unit tests. The provided `tests/test_ai_smoke.py` passes unmodified.

Suite	Coverage
<code>src/</code> (whole SE layer, combined)	88.59%
Provided <code>ai/</code> smoke tests	passing

We deliberately left a handful of exception-handling branches in the AI wrapper only partially covered — specifically, the rare interleaving where a timeout occurs on the very last retry attempt — since reliably forcing that exact timing in a test would require flaky, timing-dependent mocking that would cost more in test-suite reliability than it would gain in coverage percentage. One mock we had to redesign mid-project was our tracing test's mock tracer: a plain `Mock()` does not support the `with ... as span:` context-manager protocol our tracing code uses, so it silently failed to catch a real bug until we switched to `MagicMock()`.

6.2 Notable tests

- (i) **Happy-path end-to-end:** a test registers lost and found items fully offline and confirms a correct top-k match is returned.
- (ii) **Concurrency test:** a test in our suite exercises `asyncio.gather` with one task raising among several concurrent tasks, and confirms the batch runner returns the successful results alongside the failure rather than discarding the entire batch when one item fails — an important distinction, since losing an entire batch to one bad item would contradict the graceful-degradation principle the rest of the system is built around.
- (iii) **Error-path test that surfaced a real bug:** a test simulating a provider that raises a plain `Exception` (rather than the documented `ProviderError`) originally failed, because that exception leaked past our error handling as a raw, unhandled exception instead of the clean `AIServiceError` every caller expects. This directly led us to add a final catch-all exception handler at the AI-wrapping boundary.

What we wish we'd tested: the rate limiter's behaviour under sustained concurrent load against a genuinely live per-minute limit — we unit-tested the token-bucket logic in isolation, but never integration-tested it against real concurrent 429s the way we did (usefully, if unintentionally) for the retry logic against Gemini's live daily quota.

7 Deployment & Reproducibility

7.1 Docker image

```
docker build -t lost-and-found .
docker run --env-file .env -p 8000:8000 lost-and-found
```

We use a multi-stage build: a build stage compiles dependencies, and the runtime stage ships only what is needed to run the application.

7.2 Configuration

Variable	Required?	If missing
LLM_PROVIDER	yes	Factory raises a clear configuration error
LLM_MODEL	yes	Same
GOOGLE_API_KEY (or provider-matching key)	yes	ProviderError at first AI call
EMBEDDING_PROVIDER / EMBEDDING_MODEL	yes	Same as above
DATABASE_URL	yes	Storage layer fails to connect at startup
LOG_LEVEL	no (default INFO)	Defaults silently
MAX_IMAGE_SIZE_MB	no	Falls back to a sane default
OTEL_EXPORTER_OTLP_ENDPOINT	no	Tracing defaults to console output only

8 Limitations & Future Work

- Our retry policy treats every `ProviderError` identically, but a per-minute rate limit and a per-day quota exhaustion are different failure classes — retrying the latter within the same day wastes remaining quota for no benefit. With another week, we would classify failures and skip retries for quota-exhaustion errors specifically.
- Timeouts are thread-based: Python cannot forcibly kill an in-flight thread, so a timed-out call keeps running in the background even after we give up waiting on it.
- Token counts recorded for cost telemetry are estimated via a character-count heuristic, not the provider's actual reported usage, since the provider contract does not expose real token counts to our wrapper.
- The embedding cache has no eviction policy or size cap; a very long-running process would grow memory usage unboundedly.
- If the embedding model changes mid-deployment, previously-stored embeddings silently become incompatible with newly-computed ones for similarity comparison, with no automatic detection.

The worst design decision, in hindsight, was not distinguishing failure classes in the retry policy from the start — we only discovered the gap by hitting it live against a real provider, rather than by design. We did not get to load-test the rate limiter under sustained real concurrent traffic. The scenario most likely to break the system on a busy day is a burst of registrations that collectively exceed the daily

provider quota partway through, silently degrading match quality for the rest of that day without an obvious user-facing warning.

9 Tools & Acknowledgements

Sadiq Sadiqov (Persona A): For the database and storage modules (`src/storage/`), LLM assistants were used to design SQL schema constraints, write Alembic migration scripts, and configure pgvector indexing parameters. All database queries and schema definitions were validated against local PostgreSQL instances.

Ülkər Kərimova (Persona B): For the API, concurrency, VLM pipeline, and telemetry modules (`src/services/concurrency/`, `src/telemetry/cost.py`, and `api.py`), LLM assistants were utilized to iteratively refine asynchronous batch processing logic and FastAPI schema validation models. Critical failure cases were explicitly debugged and fixed: handling upstream HTTP 429 quota exhaustion gracefully, ensuring non-blocking execution during heavy Gemini calls, and resolving raw exception leaks in Pydantic models. All generated code was verified using local pytest execution.

Oğuz Həsənlı (Persona C): For the AI-wrapping and telemetry module (`src/services/ai_service.py`, `src/telemetry/`), Claude drafted the retry/timeout/telemetry wrapper iteratively across many review rounds. We required and verified fixes for: a fake timeout that only measured elapsed time after the fact instead of enforcing one; a silent-exception leak that let non-ProviderError failures crash uncaught; an OpenTelemetry exporter misconfigured to retry against a nonexistent local collector; and a batch-runner bug (found during integration review) that discarded an entire batch of results when one item failed. Every fix was verified by rerunning the real test suite, not accepted on the assistant's claim alone.

References

- [1] Google Gemini API documentation, <https://ai.google.dev/gemini-api/docs>
- [2] tenacity library, <https://tenacity.readthedocs.io/>
- [3] FastAPI documentation, <https://fastapi.tiangolo.com/>
- [4] pydantic-settings documentation, https://docs.pydantic.dev/latest/concepts/pydantic_settings/
- [5] OpenTelemetry Python documentation, <https://opentelemetry.io/docs/languages/python/>

A Appendix — Reproducing the benchmark

Exact commands to reproduce the Section 3.2 benchmark.

```
$ git clone https://github.com/0ghuz20/lost-and-found-project
$ cd lost-and-found-project
$ cp .env.example .env # fill in real keys
$ docker build -t lost-and-found .
$ docker run --env-file .env lost-and-found python scripts/bench.py --N 20
```

End of report — thank you for reading.